OpenAI Codex

Codex CLI 使用教程

从第一次上手，到形成稳定工作流

这是一份面向初学者的中文讲义版教程。它不追求“参数大全”，而是把安装、提问、权限、审改动、自动化和排错串成一条真正能落地的使用路径。

适合读者

会一点命令行，但第一次系统使用 Codex CLI 的开发者。

阅读重点

真实工作流、提示词写法、权限边界、Git 与 CI（Continuous Integration，持续集成）配合。

交付目标

看完就能独立完成一轮“理解项目、定位问题、做小改动、复核结果”。

本文根据本地 `codex --help` 输出与 OpenAI 官方 Codex 文档整理；请以你当前安装版本为准。

目录

教程大纲 6

1 Codex CLI 是什么 8

1.1 先看一张工作流程图 8

1.2 一个很实际的场景 8

2 安装前准备 9

2.1 你至少要准备什么 9

2.2 安装前建议先确认目录 9

2.3 再确认一下你的使用目标 10

3 安装、登录与首次验证 10

3.1 先装上 10

3.2 登录有两条主路 11

3.3 方式一：用 ChatGPT 登录 11

3.4 方式二：用 API key 登录 11

3.5 无浏览器环境怎么办 11

3.6 第一次验证，不要跳过 12

3.7 升级也很简单 12

4 基本使用流程 12

4.1 典型使用过程步骤图 13

4.2 第一次提问，建议这样说 13

4.3 追问时，尽量把范围收紧 13

5 常见命令与操作方式 14

5.1 先看一张分类图 14

5.2 交互式入口：`codex` 14

5.3 最基础的帮助与状态 14

5.4 非交互执行：`codex exec` 14

5.5 代码评审：`codex review` 15

5.6 恢复、分叉、应用结果 15
5.7 两个高频参数 15

6 权限、沙箱与审批机制 16
6.1 先记住三个沙箱级别 16
6.2 再记住三个审批策略 16
6.3 最常见的组合 17
6.4 为什么很多人推荐从 `workspace-write` 开始 17
6.5 什么时候会要求你批准 17
6.6 一个判断思路图 18

7 实用斜杠命令与会话控制 18
7.1 最值得先记的几条 18
7.2 `/status`：先确认它到底在什么状态 19
7.3 `/permissions`：会话里随时改权限 19
7.4 `/diff` 和 `/review`：改完一定要看 19
7.5 `/plan` 和 `/init`：一个管当前任务，一个管长期规则 19

8 最小可运行示例 20
8.1 示例目标 20
8.2 一轮最小示例 20
8.3 最后别忘了做一次会话内检查 20

9 高质量提示词模板库 21
9.1 模板一：看懂陌生项目 21
9.2 模板二：定位某个功能 21
9.3 模板三：修一个明确的 bug 22
9.4 模板四：做一个小改动，但限制范围 22
9.5 模板五：做代码审查 22
9.6 模板六：让它先采访你 22

10 几个典型使用场景 23
10.1 场景一：接手陌生项目 23
10.2 场景二：定位某个功能 23

10.3 场景三：解释报错 23
10.4 场景四：改一个小功能 23
10.5 场景五：提交前做一次代码审查 24

11 和 Git、测试、非交互模式的配合 24
11.1 和 Git 的最小配合方式 24
11.2 改完以后，别只看它说了什么 24
11.3 如果你知道测试命令，最好明确告诉它 25
11.4 把长期规则写进 `AGENTS.md` 25
11.5 非交互执行：什么时候该用 `codex exec` 25
11.6 CI 里怎么认证 26
11.7 为什么 `exec` 默认要求 Git 仓库 26

12 常见错误与排查思路 26
12.1 排查思路图 27
12.2 问题一：命令找不到 27
12.3 问题二：明明在项目里，却读不到对的内容 27
12.4 问题三：回答太空泛 27
12.5 问题四：它总在问你要不要批准 28
12.6 问题五：脚本里跑不通 28

13 使用建议与最佳实践 28
13.1 建议一：任务要具体 29
13.2 建议二：先小后大 29
13.3 建议三：让它输出结构化结果 29
13.4 建议四：让它知道怎么验证 29
13.5 建议五：复杂任务先计划 29
13.6 建议六：把重复规则沉淀下来 30

14 新手最容易踩的坑 30

15 3 分钟速读总结 31

16 常用命令速查表 31

17 新手避坑清单 **32**

18 附录：从零到一演示页 **32**

18.1 目标 32

18.2 第 1 步：把公开仓库拉到本地 33

18.3 第 2 步：第一次进入会话 33

18.4 第 3 步：定位一个具体点 33

18.5 第 4 步：做一个低风险改动 33

18.6 第 5 步：复核改动 34

18.7 第 6 步：完成一轮最小闭环 34

19 附录：好提示词 vs 坏提示词 **35**

19.1 一个简单判断法 35

19.2 最常见的坏味道 36

20 附录：常见场景小抄 **36**

20.1 场景一：接手项目 36

20.2 场景二：修 bug 36

20.3 场景三：补测试 37

20.4 场景四：做 review 37

21 附录：CI 集成示例 **37**

21.1 最小 shell 示例 37

21.2 一个适合 CI 的思路 38

21.3 GitHub Actions 风格示例 38

21.4 CI 里最容易踩的坑 38

22 附录：真实排错案例 **39**

22.1 案例背景 39

22.2 第一轮，不要急着让它直接修 39

22.3 第二轮，把范围压到文件级别 40

22.4 第三轮，让它给最小修复方案 40

22.5 这类案例里，Codex 最值得帮你的地方 41

22.6 如果它一上来就想改很多文件怎么办 41

22.7 这页案例真正想教你的东西 41

23 附录：练习题页 42

23.1 练习一：找到阅读入口 42

23.2 练习二：定位一个功能 42

23.3 练习三：做一个低风险小改动 43

23.4 练习四：做一次 review 43

23.5 练习五：写一条更好的提示词 43

24 附录：AGENTS.md 示例 44

24.1 什么样的内容最值得写进去 44

24.2 一份适合中小型项目起步的样板 45

24.3 如果你的仓库更复杂，可以再加什么 46

24.4 写 AGENTS.md 的一个实用原则 46

25 附录：版本变更提示框 47

25.1 最推荐的检查顺序 47

25.2 哪些地方最容易发生版本差异 48

26 FAQ 48

26.1 Q1：如果上下文窗口超了，应该怎么办？ 48

26.2 Q2：它回答得很空泛，是模型不行吗？ 49

26.3 Q3：什么时候该开新会话，而不是一直接着问？ 49

26.4 Q4：它改完代码后，我最少要做什么检查？ 49

27 官方资源与版本说明 49

27.1 当教程和你本地行为不一致时怎么办 50

28 一页式总结页 51

教程大纲

1. 先搞明白：Codex CLI 到底是干什么的
2. 安装前准备：别一上来就乱装
3. 安装、登录与首次验证：先让它真实可用
4. 基本使用流程：第一次怎么顺手
5. 常见命令与操作方式：交互、恢复、评审、应用改动
6. 权限、沙箱与审批机制：为什么有些命令能跑，有些要你批准
7. 实用斜杠命令：在会话里快速切模式和看状态
8. 最小可运行示例：从进入目录到完成一轮任务
9. 高质量提示词模板库：直接拿来改
10. 典型使用场景：什么时候最值得用
11. 和 Git、测试、非交互模式的配合：从日常开发到脚本化
12. 常见错误与排查思路：真出问题时先查哪里
13. 使用建议与最佳实践：把偶尔好用变成稳定好用
14. 新手最容易踩的坑：提前躲开
15. 3 分钟速读总结
16. 常用命令速查表
17. 新手避坑清单
18. 附录：从零到一演示页
19. 附录：好提示词 vs 坏提示词
20. 附录：常见场景小抄
21. 附录：CI 集成示例
22. 附录：真实排错案例
23. 附录：练习题页
24. 附录：AGENTS.md 示例

25. 附录：版本变更提示框

26. 官方资源与版本说明

27. 一页式复习总结

1 Codex CLI 是什么

这一节学完你能做什么

学完这一节，你会知道 Codex CLI 适合干什么、不适合干什么，以及为什么很多人把它当成“终端里的代码搭档”。

先用一句最朴素的话解释：**Codex CLI 是一个跑在终端里的 AI 编码代理**。你在项目目录里和它对话，它会结合当前代码、目录、命令行环境，帮你读代码、改代码、跑命令、解释报错、做代码审查。

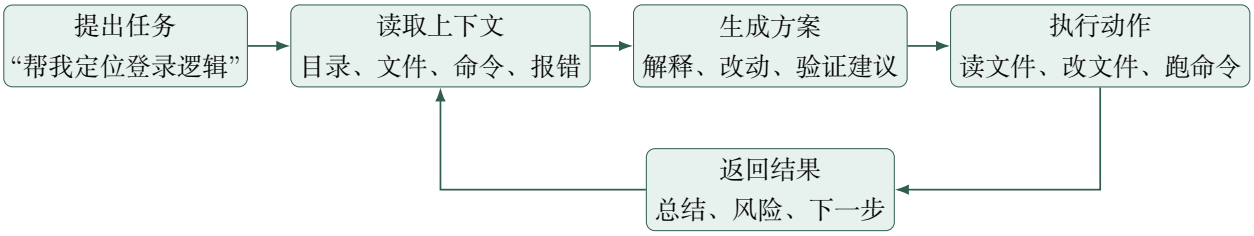
它最适合的事，通常是下面四类：

- 接手陌生项目时，先帮你找入口、找主流程、给阅读顺序。
- 你知道要改哪里，但不想自己先翻几十个文件。
- 你遇到构建失败、测试失败、脚本报错，想先得到一个靠谱的排查顺序。
- 你要补文档、补测试、改小功能，希望先出一个可用初稿。

它不太适合的事，也要提前说清楚：

- 你完全没有目标，只想“让它自动把整个项目做完”。
- 你准备把高风险命令、删除操作、线上变更全交给它，不做复核。
- 你把需求判断、架构取舍、业务责任也一起外包给它。

1.1 先看一张工作流程图



1.2 一个很实际的场景

比如你打开一个陌生仓库，里面有十几个目录。你现在既不知道入口文件在哪，也不知道应该从前端、后端还是脚本开始看。你可以让 Codex CLI：

- 先扫目录结构；

- 再指出入口文件和关键目录；
- 然后按新手视角给阅读顺序；
- 最后在你确认范围后做小改动。

提示

把 Codex CLI 理解成“既能看代码也能动手，但需要你给目标和边界的工程搭档”，这个心智模型最稳。

小结

Codex CLI 的价值，不是神奇自动化，而是把“理解项目、修改项目、解释结果、验证改动”放到一个终端工作流里。

2 安装前准备

这一节学完你能做什么

学完这一节，你会知道在安装前至少要确认哪些东西，避免工具装上了，却因为目录、账号或系统环境卡住。

很多新手的问题不是“不会装”，而是“装完以后不知道为什么不能用”。所以安装前先确认下面几件事。

2.1 你至少要准备什么

- 一台能正常使用终端的电脑。
- 一个你愿意拿来练手的项目目录，最好不是最核心的正式仓库。
- 可用的认证方式：ChatGPT 账号登录，或者 OpenAI API key。
- 基本的命令行习惯，比如会用 `cd`、`ls`、`pwd`。

2.2 安装前建议先确认目录

```
pwd
ls
```

这两条命令看起来很基础，但足够帮你避免很多低级错误：

- 先确认你现在是不是站在目标项目里；

- 再确认这个目录到底有哪些文件和子目录。

注意

如果你还没搞清楚“当前目录是不是目标项目”，先别急着让 Codex 改东西。这是新手最常见的坑之一。

2.3 再确认一下你的使用目标

安装前最好先回答自己一个问题：你现在主要是想用它做什么？

- 只是想交互式地看项目、改小功能。
- 想把某些固定任务做成脚本或 CI（Continuous Integration，持续集成）。
- 想在本地终端里配合 Git 做评审和修复。

不同目标会影响你后面更常用的功能。比如日常开发更常用 `codex`、`/diff`、`/review`；脚本化更常用 `codex exec`。

小结

安装前先确认三件事：目录、认证方式、使用目标。先把“在哪里用、怎么登、打算拿它做什么”说清楚，后面会顺很多。

3 安装、登录与首次验证

这一节学完你能做什么

学完这一节，你会知道怎么把 Codex CLI 真的装起来、登录进去，并完成第一次最小验证。

3.1 先装上

OpenAI 官方 Codex CLI 页面当前给出了 `npm` 安装入口。下面用最常见的方式举例：

```
npm i -g @openai/codex
```

装完后可以先确认命令是否存在：

```
codex --version
codex --help
```

如果 `--help` 能正常输出，说明至少两件事成立了：

- CLI 已经安装成功；
- 当前终端能找到 `codex` 这个命令。

3.2 登录有两条主路

官方文档当前写得很明确：Codex 支持两种 OpenAI 登录方式。

- 用 ChatGPT 登录：适合日常交互式使用，是 CLI 默认的登录路径。
- 用 API key 登录：适合按 API 计费的使用方式，也常用于自动化和 CI。

3.3 方式一：用 ChatGPT 登录

最直接的方式通常就是先跑：

```
codex
```

第一次运行时，CLI 会提示你登录。按官方文档，默认会走 ChatGPT 登录流程，浏览器会打开登录页，完成后再回到终端。

如果你想显式发起登录，也可以：

```
codex login
codex login status
```

3.4 方式二：用 API key 登录

如果你更希望按 API 使用方式来计费，或者你就是打算后面接脚本，那 API key 登录很实用。本地 CLI 帮助里给出了这类写法：

```
printenv OPENAI_API_KEY | codex login --with-api-key
```

它的意思很简单：把环境变量里的 key 从标准输入喂给登录命令。

注意

别把 API key 直接写进教程截图、聊天记录或仓库文件里。环境变量是最基本也最稳妥的做法。

3.5 无浏览器环境怎么办

如果你在远端机器、容器或者没有图形界面的环境里工作，官方文档给了两条思路：

- 设备码登录：运行 `codex login --device-auth`，按提示在浏览器里完成一次性验证。
- 在本地登录后，把认证缓存复制过去。

官方文档还特别提醒：如果你使用文件形式缓存认证信息，`~/.codex/auth.json` 需要当成密码对待，绝对不要提交到仓库。

3.6 第一次验证，不要跳过

建议按下面顺序做一次最小验证：

1. `codex --help` 能输出；
2. `codex login status` 能确认已登录；
3. 进入一个项目目录后，运行 `codex`；
4. 提一个最小任务，看看它能不能正确读取项目结构。

3.7 升级也很简单

官方 CLI 页面当前给出的 npm 升级方式是：

```
npm i -g @openai/codex@latest
```

提示

新版本发布会比较频繁。你如果发现教程里的某个选项和本地帮助不一致，优先相信你机器上的 `codex --help`。

小结

这一章最重要的不是“背命令”，而是形成一个稳定顺序：安装、登录、验证。只要这三步走通，后面的使用成本会大幅下降。

4 基本使用流程

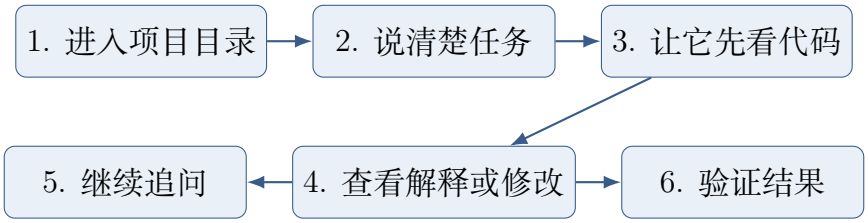
这一节学完你能做什么

学完这一节，你会知道第一次真正使用 Codex CLI 的标准节奏，至少能独立完成一轮“提任务 -> 看结果 -> 继续追问 -> 做验证”。

对新手来说，最容易上手的不是复杂参数，而是一个简单的工作流：

1. 进入项目目录；
2. 用一句清楚的话描述任务；
3. 先让它读上下文；
4. 看它的解释、计划或改动建议；
5. 不满意就继续追问；
6. 改完以后自己做验证。

4.1 典型使用过程步骤图



4.2 第一次提问，建议这样说

比起一句“帮我看看这个项目”，下面这种说法通常更有效：

```
请先看一下当前项目的目录结构，告诉我入口文件、主要模块，以及新手应该先看哪几个文件。
```

4.3 追问时，尽量把范围收紧

如果第一轮结果太泛，你可以继续这样问：

```
请不要泛泛介绍，重点解释登录相关代码在哪、数据流怎么走，以及我应该先从哪个文件开始读。
```

注意

“帮我优化一下”“帮我改好”这种说法太宽，容易得到看起来很忙、但不够落地的结果。新手阶段尽量把目标说具体。

小结

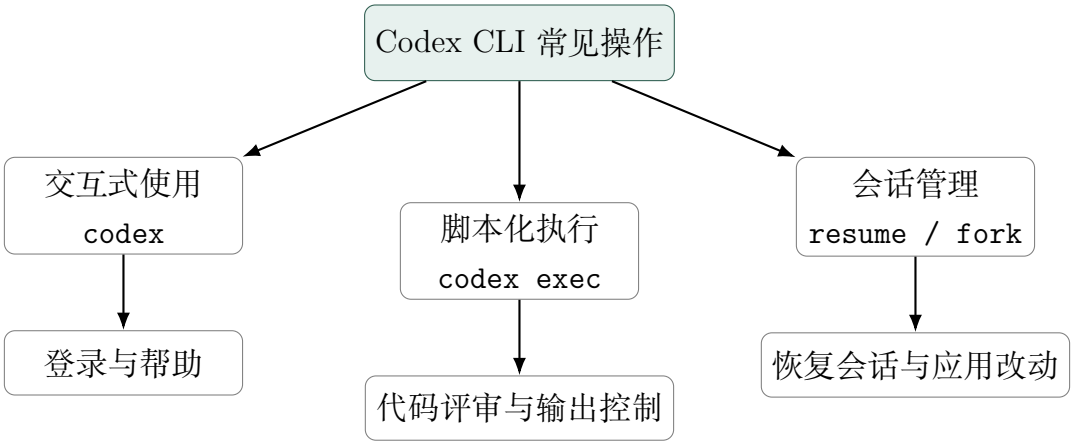
第一次用 Codex CLI，不求一步到位。先学会把任务说清楚，再学会继续追问，这比背一堆参数更重要。

5 常见命令与操作方式

这一节学完你能做什么

学完这一节，你会知道日常最常用的命令有哪些，各自适合什么场景。

5.1 先看一张分类图



5.2 交互式入口：codex

```
codex
```

这是最常见的入口。适合下面这些场景：

- 你要围绕当前项目连续提问；
- 你想边看解释边调整方向；
- 你希望它先分析，再改动，再复盘。

5.3 最基础的帮助与状态

```
codex --help
codex --version
codex login status
```

忘了支持什么功能、当前版本是什么、是否已经登录，先看这些，比乱猜靠谱。

5.4 非交互执行：codex exec

```
codex exec "检查当前仓库里最可能导致测试失败的改动"
```

这类命令更适合脚本、流水线、一次性任务。官方文档也明确把 codex exec 定位为“脚本化或 CI 风格的非交互执行模式”。

5.5 代码评审：codex review

```
codex review --uncommitted
codex review --base main
```

从本地帮助看，review 适合两类场景：

- 你还没提交，只想先看当前未提交改动的问题；
- 你想把当前分支和某个基线分支做对比审查。

5.6 恢复、分叉、应用结果

```
codex resume --last
codex fork --last
codex apply <TASK_ID>
```

这几个命令很实用：

- resume：接着上次会话干，不用重新讲背景。
- fork：把当前思路分成两条线，试不同方案。
- apply：把某次任务产生的 diff 应到本地工作区。

5.7 两个高频参数

```
codex --search
codex --add-dir ../shared-lib
```

--search 会打开实时网页搜索能力；--add-dir 用来在主工作目录之外，再给 Codex 多一个可写目录。

提示

如果你只记三个入口，先记 codex、codex exec、codex review。这三类已经覆盖了大多数入门场景。

小结

这一章的重点不是全部记住，而是知道怎么分：交互式用 `codex`，脚本化用 `codex exec`，改动复核用 `codex review`。

6 权限、沙箱与审批机制

这一节学完你能做什么

学完这一节，你会明白为什么 Codex 有时能直接跑命令，有时会停下来问你；也会知道不同沙箱模式分别意味着什么。

这部分很重要。很多人第一次用 Codex CLI，会困惑于下面这些现象：

- 为什么它能读文件，但不能写？
- 为什么有些命令会要求你批准？
- 为什么默认情况下网络能力受限？
- 为什么 CI 里和本地交互时的习惯不一样？

答案基本都和**沙箱（sandbox）**和**审批策略（approval policy）**有关。

6.1 先记住三个沙箱级别

模式	大致含义	适合什么场景
`read-only`	只能读，不能改	你只是想看项目、看报错、做分析
`workspace-write`	可以在工作区内读写	本地日常开发，最常用
`danger-full-access`	权限非常大	只在你非常明确风险时才考虑

6.2 再记住三个审批策略

- `untrusted`：只让它自动跑已知安全的读操作，不信任的命令先问你。
- `on-request`：由代理判断什么时候需要向你请求批准。
- `never`：不弹审批，适合自动化场景，但要配合你自己选好的沙箱。

6.3 最常见的组合

官方文档里给了几组很实用的默认组合，可以直接拿来理解：

- 自动但相对稳妥：

```
--full-auto
```

它本质上等于：

```
--sandbox workspace-write --ask-for-approval on-request
```

- 只读浏览：

```
--sandbox read-only --ask-for-approval on-request
```

- 工作区可写，但不信任命令必须先批：

```
--sandbox workspace-write --ask-for-approval untrusted
```

- 完全放开：

```
--dangerously-bypass-approvals-and-sandbox
```

6.4 为什么很多人推荐从 workspace-write 开始

因为它比较符合本地开发的真实需求：

- 能改你当前项目里的文件；
- 不会默认把整个机器都放开；
- 和审批策略配合时，心里更有数。

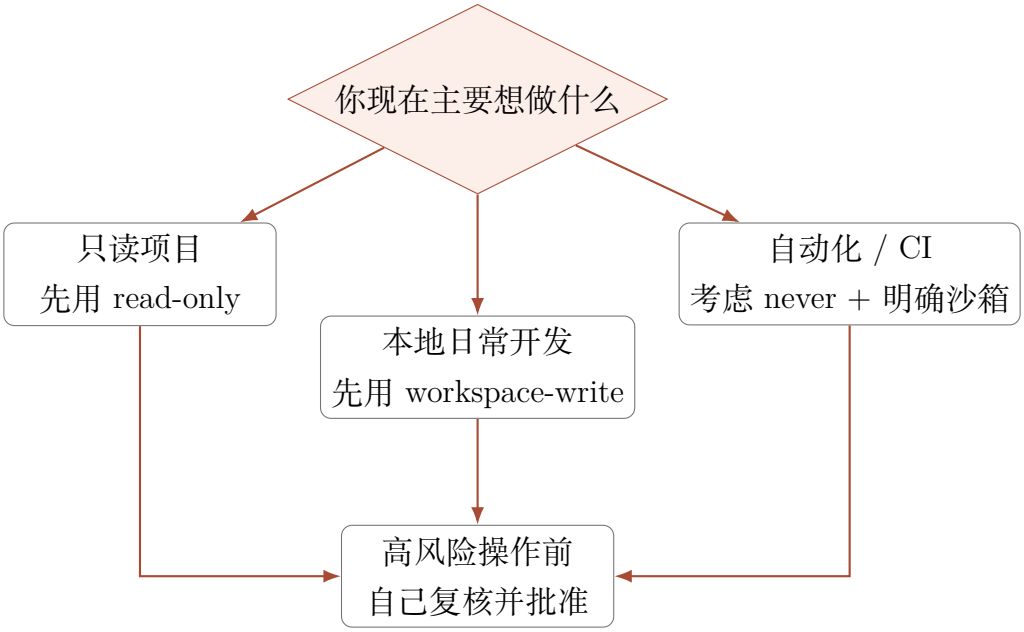
6.5 什么时候会要求你批准

官方文档明确提到，在 untrusted 模式下，Codex 会自动运行已知安全的读操作；但**会改变状态、可能有副作用、或会走外部执行路径的命令**，通常都要你批准。比如某些破坏性 Git 操作，就是典型高风险操作。

注意

--dangerously-bypass-approvals-and-sandbox 这个名字已经说得很直白了。只有在你外层环境本身就已经被严格隔离时，才考虑用它。

6.6 一个判断思路图



小结

最实用的经验是：本地开发从 `workspace-write` 起步，先别碰完全放开模式；你越清楚权限边界，Codex 用起来就越稳。

7 实用斜杠命令与会话控制

这一节学完你能做什么

学完这一节，你会知道在交互会话里有哪些高频斜杠命令，不用退出重进也能切模型、看状态、看 diff、做评审。

CLI 里很多高频操作，不一定要回到 shell 里敲完整命令。官方文档的斜杠命令页里，有几条特别适合新手先记住。

7.1 最值得先记的几条

斜杠命令	作用	最常见用途
`/status`	看当前会话状态	确认模型、审批策略、可写目录、token 使用情况
`/model`	切模型	当前任务需要更快或更深的推理时切换

斜杠命令	作用	最常见用途
/permissions	改权限模式	临时把会话改成更保守或更自动
/plan	先出执行计划	任务较大时先计划再动手
/diff	看 Git diff	改完先审一眼再提交
/review	做工作区评审	查行为风险、缺测试、潜在回归
/mention	显式提某个文件	强制把某个路径拉进上下文
/compact	压缩长会话	对话很长时瘦身，保留重点
/init	生成 AGENTS.md 脚手架	把仓库规则写成长期提示

7.2 /status：先确认它到底在什么状态

官方文档对 /status 的描述很实用：它会显示当前模型、审批策略、可写根目录、token 使用情况。也就是说，当你觉得“它怎么突然行为不对劲”时，先跑 /status 通常最省时间。

7.3 /permissions：会话里随时改权限

你一开始可以保守一些，先读代码；真正要改时，再把权限调到更适合当前任务的模式。这样比整场会话从头到尾都开得很大更稳。

7.4 /diff 和 /review：改完一定要看

这两条适合连着用：

1. 先用 /review 让 Codex 从行为和测试角度看一下工作区；
2. 再用 /diff 看看具体改了哪些文件、哪些行。

7.5 /plan 和 /init：一个管当前任务，一个管长期规则

/plan 适合当前任务比较大时先做执行计划。/init 则会在当前目录生成 AGENTS.md 脚手架，这个文件可以把“如何跑项目、如何测试、哪些目录别碰、完成标准是什么”这些长期规则写进去。

提示

如果你经常重复说“先跑测试再改”“不要动生成文件”“改完给我验证步骤”，就该考虑用 /init 和 AGENTS.md 固化这些规则了。

小结

斜杠命令的核心价值，是让你在同一场会话里完成“看状态、改权限、切模型、看 diff、做 review”这些高频动作，而不用来回跳出会话。

8 最小可运行示例

这一节学完你能做什么

学完这一节，你至少能照着完成一轮最小体验：进入目录、发起任务、继续追问、复核结果。

8.1 示例目标

假设你刚进入一个项目目录，想知道：

- 这个项目的入口在哪；
- 哪几个文件最值得先看；
- 如果你是新手，阅读顺序应该是什么。

8.2 一轮最小示例

```
cd your-project
codex
```

进入交互后，你可以这样说：

> 请先查看当前项目结构，告诉我入口文件、核心模块、建议的阅读顺序，并尽量用新手能听懂的话解释。

如果结果不错，继续追问：

> 请进一步解释最核心的两个文件各自负责什么、它们之间怎么配合，以及我如果要改登录逻辑应该先看哪里。

8.3 最后别忘了做一次会话内检查

```
/status
```

看看当前模型、权限模式、可写目录是不是和你预期一致。如果要审改动，再接：

```
/diff
```

注意

如果它只给你空泛总结，没有落到具体文件名、模块名和路径，那就继续追问，不要直接收工。

小结

最小示例的目标不是展示高级能力，而是让你真正建立起“我已经会跑通第一轮”的感觉。

9 高质量提示词模板库

这一节学完你能做什么

学完这一节，你会拿到一组可以直接复制、稍改就能用的模板，而不是每次临场现编。

很多时候，Codex 用得好不好，不在于你是不是写了“高级提示词”，而在于你有没有说清楚**目标**、**范围**、**约束**、**输出格式**。

9.1 模板一：看懂陌生项目

```
请先从新手视角介绍当前项目。
重点告诉我：
1. 入口文件在哪里
2. 核心目录分别负责什么
3. 主流程怎么走
4. 建议我按什么顺序阅读
请尽量引用具体文件路径，不要泛泛而谈。
```

9.2 模板二：定位某个功能

```
请帮我定位“登录”相关代码。
我想知道：
1. 入口在哪里
2. 涉及哪些文件
3. 调用链大概怎么走
4. 如果我要改登录失败提示，应该先看哪里
```

9.3 模板三：修一个明确的 bug

```
我遇到了这个报错，请先不要急着改代码。
请按下面顺序回答：
1. 最可能的原因
2. 建议的排查顺序
3. 最值得先看的文件
4. 如果要修，最小改动方案是什么
```

9.4 模板四：做一个小改动，但限制范围

```
请只修改提示文案，不要改业务逻辑。
修改前先告诉我你准备改哪些文件。
修改后请总结：
1. 改了哪些文件
2. 每个文件改了什么
3. 我应该怎么验证
```

9.5 模板五：做代码审查

```
请像代码 reviewer 一样检查当前改动。
重点关注：
1. 行为变化
2. 潜在回归
3. 边界情况
4. 缺失的测试
请先给发现的问题，再给简短总结。
```

9.6 模板六：让它先采访你

这是官方 best practices 页面里很值得学的一条思路：如果你的需求还很模糊，可以先让 Codex 反过来问你问题，把模糊目标问清楚。

```
我现在只有一个模糊想法，请先通过提问帮我把需求收紧，挑战我的假设，等目标清楚以后再开始修改代码。
```

提示

真正高质量的提示词，不是写得花，而是让 Codex 明白“你到底要它做什么、不许它做什么、最后要怎么交付结果”。

小结

写提示词时，尽量补齐四要素：目标、范围、限制、输出格式。这个习惯比背模板更重要。

10 几个典型使用场景

这一节学完你能做什么

学完这一节，你会更容易判断什么时候值得用 Codex CLI，什么时候自己手动做反而更快。

10.1 场景一：接手陌生项目

你最需要的不是“全量介绍”，而是“给我一个阅读入口”。

```
请按新手视角介绍这个项目，重点说入口、核心目录、主要数据流，并给出推荐阅读顺序。
```

10.2 场景二：定位某个功能

```
请帮我定位上传功能相关代码，告诉我涉及哪些文件、入口在哪里、调用链大概是什么。
```

10.3 场景三：解释报错

```
这是我刚遇到的报错。请先解释最可能的原因，再给我一个排查顺序，最后告诉我应该先看哪些文件。
```

10.4 场景四：改一个小功能

```
请把这里的提示文案改得更清楚，但不要改业务逻辑。修改后请说明改了哪些文件，以及如何验证。
```

10.5 场景五：提交前做一次代码审查

```
/review
/diff
```

这是很典型的一套动作：先让它帮你找风险，再自己看差异细节。

小结

Codex CLI 最值钱的地方，往往不是帮你“一口气写完整个功能”，而是大幅缩短“理解、定位、修改、复核”这几步之间的时间。

11 和 Git、测试、非交互模式的配合

这一节学完你能做什么

学完这一节，你会知道如何把 Codex CLI 接进真实开发流程，而不是只停留在聊天式体验。

11.1 和 Git 的最小配合方式

你可以把一轮工作想成这样：

1. 先让 Codex 解释或修改；
2. 再用 `/diff` 或 Git diff 看差异；
3. 再让它做 `/review` 或 `codex review`；
4. 最后自己跑测试、决定是否提交。

如果你想用命令行方式做审查，本地帮助里有这些入口：

```
codex review --uncommitted
codex review --base main
```

11.2 改完以后，别只看它说了什么

真正重要的是验证：

- 它改的文件是不是你预期的文件；
- 它有没有顺手碰到不该碰的模块；
- 构建、测试、关键路径是不是还过得去。

11.3 如果你知道测试命令，最好明确告诉它

官方 best practices 里有一个很关键的提醒：如果你不告诉 Codex 该怎么构建、怎么测试，它就不容易验证自己的工作。所以你可以在提示里直说：

这里顺手解释一下：`lint` 一般指**静态检查**，主要用来提前发现格式、风格和一些明显的代码问题。

```
改完以后请运行项目已有的测试命令。如果你不确定，先搜索仓库里现有的 test / build / lint 脚本再决定。
```

11.4 把长期规则写进 AGENTS.md

官方 best practices 明确建议，把重复出现的规则沉淀到 `AGENTS.md`，而不是每次都重新说一遍。一个好用的 `AGENTS.md` 往往会包括：

- 仓库结构和重点目录；
- 构建、测试、lint 命令；
- 工程约定和禁止事项；
- “算完成”的标准和验证方式。

你可以先用：

```
/init
```

生成脚手架，再按团队真实习惯补充内容。

11.5 非交互执行：什么时候该用 codex exec

如果你想把某些工作做成脚本，或者在 CI 里跑，`codex exec` 很适合。比如：

```
codex exec "总结当前仓库最重要的三个风险点"
```

它还支持把最终消息写到文件里：

```
codex exec "生成本次改动总结" -o summary.txt
```

如果你要机器可读输出，可以加：

```
--json
```

如果你要严格结构化结果，可以再加：

```
--output-schema
```

11.6 CI 里怎么认证

官方 non-interactive 文档明确建议：在 CI 里优先用 API key，并通过 `CODEX_API_KEY` 注入。它还特别指出：`CODEX_API_KEY` 这个环境变量**只在 `codex exec` 里支持**。

```
CODEX_API_KEY=<api-key> codex exec --json "triage open bug reports"
```

11.7 为什么 exec 默认要求 Git 仓库

官方文档也写得很清楚：`codex exec` 默认要求在 Git 仓库内运行，这是为了降低破坏性改动风险。如果你非常确定环境安全，才考虑：

```
codex exec --skip-git-repo-check "..."
```

注意

把一项任务做成自动化之前，先手工把它跑稳定。官方 best practices 也明确不建议过早把不稳定流程自动化。

小结

从“能聊天”到“能进工作流”，关键就三件事：看 diff、跑测试、把重复规则沉淀到 `AGENTS.md`。而脚本化场景则优先考虑 `codex exec`。

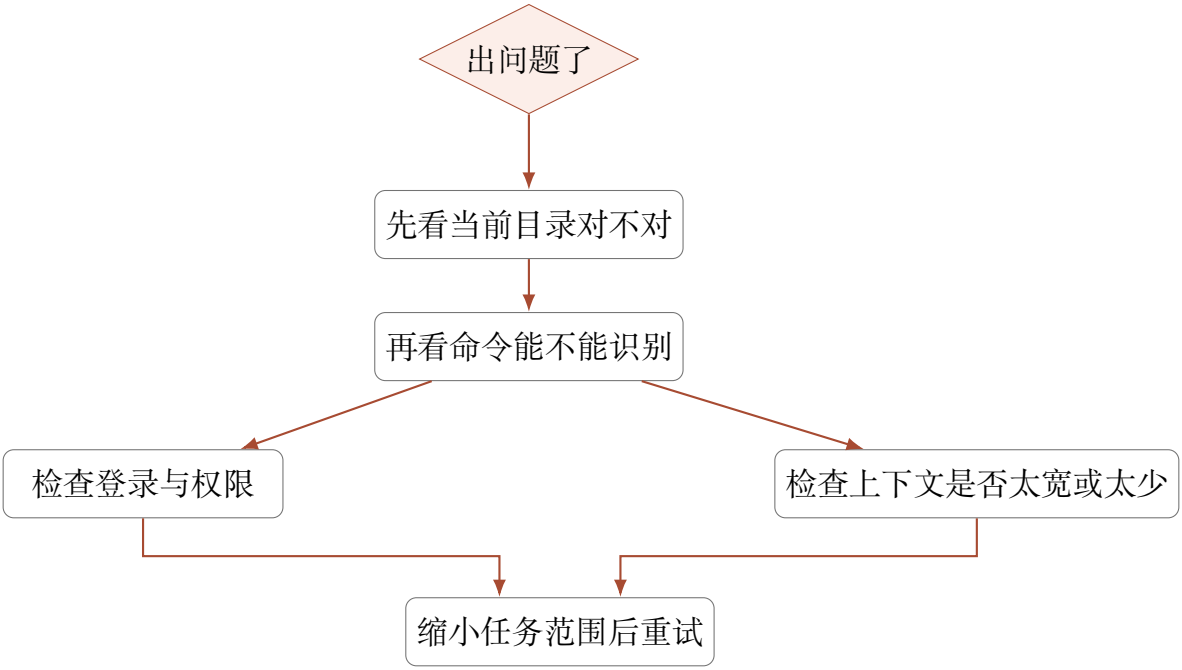
12 常见错误与排查思路

这一节学完你能做什么

学完这一节，你遇到问题时至少知道先查什么，不会一报错就完全没方向。

最常见的问题，通常不是模型不聪明，而是环境、目录、登录、权限、上下文这几类基础问题。

12.1 排查思路图



12.2 问题一：命令找不到

```
command not found: codex
```

优先检查：

- 是否真的安装成功；
- 当前 shell 能不能找到 `codex`；
- 终端会话是否需要重新打开。

12.3 问题二：明明在项目里，却读不到对的内容

先确认你是不是在正确目录：

```
pwd
ls
```

如果目录没问题，再看你是不是把任务说得太宽了。

12.4 问题三：回答太空泛

大概率不是它完全不会，而是你的任务太宽。解决办法通常是：

- 缩小目标；
- 指定文件、模块或功能；
- 明确输出格式，比如“列出文件路径 + 每个文件一句解释”。

12.5 问题四：它总在问你要不要批准

这不一定是坏事。多数时候只是当前审批策略较保守，或者它准备执行的命令本来就不属于安全读操作。

你可以先用：

```
/status
/permissions
```

看当前会话权限，再决定要不要调模式。

12.6 问题五：脚本里跑不通

如果你在 CI 或脚本里用 `codex exec`，优先确认：

- 认证是不是用的 `CODEX_API_KEY`；
- 当前目录是不是 Git 仓库；
- 是否需要 `--output-last-message` 或 `--json` 这类更适合自动化的输出方式。

注意

高风险操作、删除操作、覆盖性修改，一定要自己复核。AI 可以帮你提速，但不能替你承担最终判断责任。

小结

排查时优先看五件事：目录、安装、登录、权限、任务范围。先把这五件事排清楚，通常已经能解决大部分问题。

13 使用建议与最佳实践

这一节学完你能做什么

学完这一节，你会知道怎样把 Codex CLI 用得更稳、更省时间，而不是越用越乱。

13.1 建议一：任务要具体

尽量少说“帮我优化一下”，多说“帮我解释这个目录”“帮我定位登录逻辑”“只改文案，不改业务逻辑”。

13.2 建议二：先小后大

别一上来就扔超大任务。更稳妥的顺序通常是：

1. 先解释；
2. 再定位；
3. 再计划；
4. 最后再修改。

13.3 建议三：让它输出结构化结果

比如直接要求：

- 按文件列表输出；
- 每个文件一句话解释；
- 先结论，再细节；
- 最后给验证建议。

13.4 建议四：让它知道怎么验证

官方 best practices 明确提醒，不要让代理在不知道怎么构建、测试、lint 的前提下瞎试。所以最好在仓库里维护好 `AGENTS.md`，或者在提示中说明验证方式。

13.5 建议五：复杂任务先计划

当任务比较长、改动面比较广时，先用 `/plan` 或者直接让它先出计划，再决定是否实施。这样能明显减少“上来就改，结果越改越散”的问题。

13.6 建议六：把重复规则沉淀下来

如果你经常重复同样的话，就说明这条规则应该搬进 `AGENTS.md`、技能文件或配置，而不是每次重新输入。

提示

把 Codex CLI 当成“会先打草稿、再陪你迭代”的工程搭档，比把它当成“全自动代打工具”稳定得多。

小结

最佳实践就一句话：任务说清楚，范围控住，验证补上，重复规则沉淀下来。

14 新手最容易踩的坑

这一节学完你能做什么

学完这一节，你会提前知道哪些地方最容易翻车，尤其适合第一次正式使用前快速扫一遍。

1. 没确认当前目录，就开始让它读代码或改文件。
2. 任务描述太宽，结果很空，还误以为是工具不行。
3. 一上来就给太大权限，自己却没想清楚风险。
4. 看见它改完了就直接接受，没有自己看 diff 和跑验证。
5. 把“解释、定位、修改、验证”四件事全揉进一个超长需求里。
6. 在自动化场景里直接照搬交互式习惯，没有单独处理认证和输出。
7. 有重复规则却不写进 `AGENTS.md`，导致每次都要重新解释。

注意

新手最大的问题通常不是不会敲命令，而是没有形成稳定节奏：先确认，再限制范围，再验证结果。

小结

避坑的关键不是记住一堆例外，而是养成流程：确认目录、收紧任务、看状态、审改动、做验证。

15 3 分钟速读总结

如果你现在只想记住最重要的几点

1. 把 Codex CLI 当成终端里的代码搭档，而不是全自动代打工具。
2. 安装后先做三件事：`codex --help`、登录、最小任务验证。
3. 日常开发多用 `codex`，脚本化多用 `codex exec`，提交前别忘 `/diff` 和 `/review`。
4. 本地开发优先从 `workspace-write` 这类中间档权限开始，不要急着全放开。
5. 高质量提示词的关键是：目标、范围、限制、输出格式。
6. 真正让 Codex 稳定好用的，不是一次神 prompt，而是 `AGENTS.md`、清晰测试方式和稳定工作流。

16 常用命令速查表

命令/操作	用途	什么时候用
`codex`	进入交互式会话	日常本地开发、连续提问、逐步迭代时最常用
`codex --help`	查看帮助信息	忘了入口、参数、功能时先用
`codex --version`	查看版本	怀疑版本差异时先确认
`codex login`	发起登录	想显式登录而不是等首次运行触发
`codex login status`	查看登录状态	不确定当前是否已认证时
`codex exec "..."`	非交互执行	脚本化、CI、一次性任务
`codex review --uncommitted`	审查当前未提交改动	提交前快速做风险检查
`codex review --base main`	对比分支基线做评审	PR 前自查很实用
`codex resume --last`	恢复最近一次会话	不想重复讲背景时
`codex apply <TASK_ID>`	应用某次任务产出的 diff	你想把结果落到本地工作树时
`/status`	查看会话状态	确认模型、权限、可写目录
`/permissions`	调整权限模式	临时改成更保守或更自动
`/diff`	查看当前改动	改完先看一眼再提交

命令/操作	用途	什么时候用
/review	在会话内做代码审查	想先查风险和缺失测试
/plan	先做执行计划	任务较大、较复杂时
/init	生成 AGENTS.md 脚手架	想把规则沉淀到仓库里

17 新手避坑清单

- 操作前先确认你在哪个目录。
- 登录方式和使用场景要区分：交互式和 CI 不是一回事。
- 能先解释就先解释，别急着大改。
- 让它改文件时，要求说明改了哪些文件、为什么改、怎么验证。
- 会话里看不对劲，先用 /status 和 /permissions。
- 改完一定看 /diff，必要时再跑 /review。
- 有重复规则就写进 AGENTS.md，别每次重新说。
- 遇到版本差异时，以你本地 codex --help 和官方文档为准。

18 附录：从零到一演示页

这一节学完你能做什么

学完这一节，你会看到一条从克隆公开小仓库到完成一次解释、定位、修改、复核的完整路径，适合第一次照着做。

这一页的重点不是仓库本身有多复杂，而是帮你把**真实工作流顺一遍**。这里用“任意公开小仓库”来演示，你可以换成一个最小 Todo、博客或练习项目。

18.1 目标

假设你的目标很简单：

- 先看懂仓库结构；

- 再找到入口；
- 再做一个低风险改动；
- 最后复核并总结。

18.2 第 1 步：把公开仓库拉到本地

```
git clone <公开小仓库地址> demo-repo
cd demo-repo
```

这里故意不把演示绑死到某一个具体仓库名上，是为了让你能把这页直接复用到任何公开小项目。

18.3 第 2 步：第一次进入会话

```
codex
```

第一轮建议先这样问：

```
请先按新手视角查看当前项目。
告诉我：
1. 入口文件在哪里
2. 核心目录分别负责什么
3. 我应该按什么顺序阅读
请尽量引用具体文件路径。
```

18.4 第 3 步：定位一个具体点

等你拿到目录概览以后，继续把范围收紧：

```
现在请重点解释最核心的两个文件。我想知道它们分别负责什么，以及它们之间是怎么配合的。
```

18.5 第 4 步：做一个低风险改动

第一次练手，不要先碰大逻辑，优先选这类改动：

- README 说明补充；
- 文案更清楚；

- 一个明显的小 bug；
- 缺失但容易补的测试。

比如你可以这样说：

```
请只做一个低风险改动：把首页提示文案改得更清楚，不要改业务逻辑。修改前先告诉我你准备改哪些文件。
```

18.6 第 5 步：复核改动

改完以后不要立刻收工，先看：

```
/review
/diff
```

再让它补一句人话总结：

```
请总结这次改动：
1. 改了哪些文件
2. 每个文件改了什么
3. 我现在应该怎么验证
```

18.7 第 6 步：完成一轮最小闭环

这时你应该已经完成了最重要的一轮：

1. 进入仓库；
2. 读懂结构；
3. 定位关键点；
4. 做一处低风险修改；
5. 看 diff 和 review；
6. 准备验证。

提示

第一次练手时，不追求改得多，追求完整走完一轮。能稳定走完这一轮，比一次改十个文件更重要。

小结

“从零到一演示页”的核心不是仓库名，而是让你掌握一条固定路径：先理解，再定位，再小改，再复核。

19 附录：好提示词 vs 坏提示词

这一节学完你能做什么

学完这一节，你会更容易判断什么样的提问会让 Codex 输出靠谱结果，什么样的提问容易把它带偏。

场景	不太好的写法	更好的写法
看项目	帮我看看这个项目	请按新手视角介绍当前项目，重点说入口、核心目录、主流程和推荐阅读顺序
定位功能	帮我找登录	请帮我定位登录相关代码，告诉我入口、涉及文件和大致调用链
修 bug	帮我修一下这个错	这是报错信息。请先解释最可能原因，再给排查顺序，最后给最小修改方案
改代码	帮我优化一下	请只修改提示文案，不要改业务逻辑，修改后列出改动文件和验证方法
做 review	帮我看看有没有问题	请像 reviewer 一样检查当前改动，先列问题，再给简短总结，重点关注回归和缺失测试

19.1 一个简单判断法

如果一条提示词同时满足下面四件事，它通常就已经比平均水平强很多：

- 说清楚目标；
- 说清楚范围；
- 说清楚限制；
- 说清楚输出格式。

19.2 最常见的坏味道

- 只有动作，没有目标，比如“优化一下”“改一下”。
- 只有目标，没有范围，比如“把这个项目整理好”。
- 只有结果要求，没有约束，比如“修好它”，却没说能不能改架构。
- 没要求输出格式，导致结果全是大段空话。

小结

真正的区别不在“高级词藻”，而在有没有把目标、范围、限制、输出格式交代清楚。

20 附录：常见场景小抄

这一节学完你能做什么

学完这一节，你可以把常见场景当成小抄来用，遇到任务时直接照着改，不用从头想。

20.1 场景一：接手项目

提示

先问什么：入口、核心目录、主流程、阅读顺序。
推荐提示词：请按新手视角介绍当前项目，引用具体文件路径。
别忘了：先让它解释，不要一上来就改。

20.2 场景二：修 bug

提示

先问什么：最可能原因、排查顺序、最值得看的文件。
推荐提示词：先解释报错，不要急着修改。
别忘了：让它给最小修改方案，而不是上来大改。

20.3 场景三：补测试

提示

先问什么：项目现有测试框架、最接近的已有测试、这次应该补哪种测试。
推荐提示词：请先找项目里现有测试模式，再补一条风格一致的测试。
别忘了：明确要求不要顺手重构无关代码。

20.4 场景四：做 review

提示

先问什么：行为风险、边界情况、缺失测试、可能回归。
推荐动作：先 `/review`，再 `/diff`。
别忘了：要求“先列问题，再总结”，这样输出更适合 review。

小结

常见场景小抄的核心作用，是帮你减少临场组织语言的成本。越常见的任务，越值得模板化。

21 附录：CI 集成示例

这一节学完你能做什么

学完这一节，你会知道怎样把 `codex exec` 接进自动化流程，尤其是怎样组合 `--json`、`-o` 和 API key。

21.1 最小 shell 示例

```
export CODEX_API_KEY=your_api_key

codex exec --json \
  -o codex-summary.txt \
  "Review the current repository and summarize the top 3 risks"
```

这一段里，三个关键点分别是：

- `CODEX_API_KEY`：给非交互执行做认证。
- `--json`：让标准输出更适合被脚本消费。

- `-o codex-summary.txt`：把最后一条自然语言总结单独落盘。

21.2 一个适合 CI 的思路

你可以把 `codex exec` 放到下面这些位置：

- PR 创建后做风险总结；
- 每日巡检时总结仓库变化；
- 发布前做一轮只读式 review；
- 自动生成本次改动摘要。

21.3 GitHub Actions 风格示例

```
name: codex-check

on:
  pull_request:

jobs:
  codex-review:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4

      - name: Install Codex CLI
        run: npm i -g @openai/codex

      - name: Run Codex Exec
        env:
          CODEX_API_KEY: ${{ secrets.CODEX_API_KEY }}
        run: |
          codex exec --json -o codex-summary.txt \
            "Review the diff against the base branch and summarize risks"
```

21.4 CI 里最容易踩的坑

- 直接照搬交互式习惯，没有切到 `codex exec`。

- 忘了提供 API key。
- 没有考虑当前目录是不是 Git 仓库。
- 只看 stdout，没有把最终摘要单独落文件。

注意

自动化场景更适合做“总结、分类、只读式检查”这类任务。真正会改很多文件的流程，先在本地跑稳，再考虑自动化。

小结

CI 集成的关键不是把一切自动化，而是先找到最适合自动化的那一层：总结、审查、提示、归档。

22 附录：真实排错案例

这一节学完你能做什么

学完这一节，你会看到一条更接近真实开发现场的排错路径，不只是“报错了怎么办”，而是“遇到一个具体报错时，怎样让 Codex 帮你缩小范围”。

这一页故意写成案例，而不是抽象原则。因为新手最容易卡住的点，不是“不知道要排错”，而是不知道**第一句该怎么问、第二步该看什么**。

22.1 案例背景

假设你在一个前端项目里运行开发命令时，终端出现类似错误：

```
TypeError: Cannot read properties of undefined (reading 'map')
```

你目前只知道三件事：

- 页面白屏了；
- 报错和某个列表渲染有关；
- 你还没定位到具体文件。

22.2 第一轮，不要急着让它直接修

先这样问：

```
我刚遇到一个运行时报错：
TypeError: Cannot read properties of undefined (reading 'map')

请先不要直接修改代码。
先帮我做三件事：
1. 解释这个错误最常见的原因
2. 告诉我应该先检查哪些类型的文件
3. 给我一个从快到慢的排查顺序
```

这一轮的目标不是立刻修复，而是让 Codex 先帮你把问题分类。通常它会给出这种方向：

- 某个数组值还没准备好，就提前调用了 `map`；
- 接口返回为空，或者初始状态不是数组；
- 某个组件 props 没传到位。

22.3 第二轮，把范围压到文件级别

等第一轮拿到方向以后，再追问：

```
请根据当前项目代码，帮我定位最可能出现这个问题的文件。
优先找：
1. 渲染列表的组件
2. 初始化 state 的地方
3. 拉取接口数据并赋值的地方
```

这一步很关键。因为“抽象原因”到这里才真正开始落地到代码。

22.4 第三轮，让它给最小修复方案

等你已经知道大概是哪几个文件以后，再说：

```
现在请给我一个最小修复方案。
要求：
1. 先说明为什么会报错
2. 只改最必要的地方
3. 不要顺手重构
4. 修改后告诉我怎么验证
```

22.5 这类案例里，Codex 最值得帮你的地方

不是“替你秒修”，而是帮你更快完成下面这些动作：

- 把报错归类；
- 找最可能的文件；
- 给出从小到大的排查顺序；
- 限制改动范围；
- 补验证建议。

22.6 如果它一上来就想改很多文件怎么办

立刻把范围收回来：

```
请先不要做大范围修改。只给我最小修复方案，并说明如果只改一个文件，最可能改哪一个。
```

22.7 这页案例真正想教你的东西

不是这个错误本身，而是一种顺序：

1. 先解释报错；
2. 再定位文件；
3. 再最小修复；
4. 最后验证。

提示

当你不知道该不该马上让 Codex 动代码时，默认先让它解释和定位。先把问题缩小，再让它改，几乎总是更稳。

小结

真实排错案例的核心，不是记住某一种报错，而是记住排错顺序：先分类，后定位，再最小修复，最后验证。

23 附录：练习题页

这一节学完你能做什么

学完这一节，你可以把前面学到的用法真正练一遍，而不是只停留在“我看懂了”。

这一页给你几道适合新手的练习题。最好找一个公开小仓库，或者你自己的练手仓库，按顺序做。

23.1 练习一：找到阅读入口

目标：让 Codex 帮你给出项目阅读顺序。
要求：

- 问出入口文件；
- 问出核心目录；
- 问出推荐阅读顺序；
- 输出里必须包含具体文件路径。

自检点：

- 结果有没有落到具体路径；
- 结果是不是按新手视角组织的；
- 你能不能根据它给的顺序自己开始读代码。

23.2 练习二：定位一个功能

目标：找出某个功能的入口和调用链。
要求：

- 自己先选一个功能，比如登录、上传、搜索、列表渲染；
- 让 Codex 告诉你涉及哪些文件；
- 让它解释调用链，不要只列文件名。

自检点：

- 它有没有解释“文件之间怎么串起来”；
- 你能不能据此自己继续往下追。

23.3 练习三：做一个低风险小改动

目标：让 Codex 改一个你敢接受的小地方。

要求：

- 只改文案、说明或一处很小的逻辑；
- 明确要求“不要改业务逻辑”或“不要顺手重构”；
- 修改后让它列出改动文件和验证方法。

自检点：

- 改动范围有没有被控制住；
- 它有没有解释为什么这么改；
- 你有没有自己看 `/diff`。

23.4 练习四：做一次 review

目标：把 Code review 流程走一遍。

建议动作：

```
/review
/diff
```

自检点：

- 它有没有优先指出真正的问题，而不是只给表扬；
- 你能不能分清“高风险问题”和“可选优化”；
- 你有没有把它的 review 结果和实际 diff 对照起来看。

23.5 练习五：写一条更好的提示词

目标：练习把模糊需求改成清晰需求。

做法：

- 先写一句模糊请求，比如“帮我优化一下这里”；
- 再自己把它改写成包含目标、范围、限制、输出格式的版本；
- 然后分别喂给 Codex，看结果差异。

自检点：

- 改写后的结果是不是明显更落地；
- 你是不是更容易继续追问；
- 结果里是不是少了空话，多了具体文件和动作。

提示

练习题不要一次做太多。对新手来说，最有效的顺序通常是：练习一、练习二、练习三。先把“理解、定位、小改动”这三件事练熟。

小结

练习题页的重点不是难度，而是帮助你把教程里的方法变成自己的操作习惯。

24 附录：AGENTS.md 示例

这一节学完你能做什么

学完这一节，你会看到一份可以直接改成自己仓库版本的 `AGENTS.md` 样板，也会知道为什么这个文件对长期使用 Codex 很重要。

前面已经提到，`AGENTS.md` 的作用是把重复规则沉淀下来。你可以把它理解成“这个仓库专属的长期系统提示词”。这样一来，Codex 每次进入仓库时都更容易知道：

- 项目怎么跑；
- 测试怎么跑；
- 哪些目录能改，哪些别碰；
- 交付结果时你最看重什么。

24.1 什么样的内容最值得写进去

最值得沉淀的，通常是下面这些信息：

- 仓库结构和重点目录说明；
- 安装、启动、测试、lint 的标准命令；
- 改动边界，比如哪些文件是生成的、哪些目录不要动；

- 代码风格、提交前检查、输出格式要求；
- 完成任务后应该如何总结和汇报。

24.2 一份适合中小型项目起步的样板

```
# AGENTS.md

## 项目概览

- 这个仓库包含前端、后端和共享工具模块。
- 改动的首要目标是：范围小、容易评审、方便验证。
- 在做较大改动之前，优先先阅读现有代码和测试。

## 仓库结构

- `frontend/`：界面和页面代码
- `backend/`：接口处理和业务逻辑
- `shared/`：前后端共用代码
- `scripts/`：本地维护脚本
- `tests/`：自动化测试
- `docs/`：项目文档

## 工作规则

- 修改前先确认当前目录和相关文件。
- 优先做小而明确的改动，不要轻易做大范围重构。
- 除非任务明确要求，否则不要修改生成文件。
- 除非确有必要，否则不要重命名文件或移动模块。
- 如果需要高风险或破坏性改动，先说明原因再执行。

## 常用命令

- 安装依赖：`npm install`
- 启动开发环境：`npm run dev`
- 运行单元测试：`npm test`
- 运行静态检查：`npm run lint`
- 构建项目：`npm run build`

## 改动原则
```

```
- 保持现有代码风格和项目约定。
- 不要顺手修改无关文件。
- 只有在代码本身不够直观时才添加注释。
- 修 bug 时优先选择最小且正确的改动。

## 验证要求

- 改完代码后，先执行最小但相关的验证步骤。
- 如果任务影响了应用行为，尽量运行相关测试。
- 如果有必须做但无法运行的检查，请在最终总结里明确说明。

## 输出要求

- 用通俗的话总结这次改了什么。
- 如果任务不算特别简单，请列出改动过的文件。
- 说明已经做了哪些验证。
- 如果还有剩余风险或后续检查项，也请写出来。

## 默认策略

- 遇到模糊需求时，先澄清问题或先给一个简短计划。
- 遇到较大任务时，先拆成更小的步骤。
- 如果用户还在熟悉代码库，优先先解释，再修改。
```

24.3 如果你的仓库更复杂，可以再加什么

如果是团队项目，还可以继续往里补：

- 分支策略，比如不要直接动主分支；
- 数据库迁移规范；
- 前后端联调顺序；
- 哪些命令耗时长，不要默认跑；
- 哪些目录包含敏感信息或外部生成产物。

24.4 写 AGENTS.md 的一个实用原则

不要把它写成“项目说明书大全”。最实用的 AGENTS.md，通常只保留 **Codex 真的会反复用到的规则**。如果一条说明不会影响它怎么读代码、怎么改代码、怎么验证结果，那

就不一定要放进来。

提示

一个好用的判断标准是：如果你在三次任务里都重复说过同一句话，那它大概率就该进 `AGENTS.md` 了。

小结

`AGENTS.md` 的核心作用，不是展示项目背景，而是把“怎么做事”固定下来。写得越贴近真实工作流，Codex 后面越省心。

25 附录：版本变更提示框

这一节学完你能做什么

学完这一节，你会知道当教程、截图和你本地命令不一致时，应该优先检查哪些地方。

版本提示

- 安装方式可能变化：例如 npm 包名、推荐安装命令、首次登录入口。
- 登录方式可能变化：浏览器登录、设备码登录、API key 登录的推荐路径可能调整。
- 斜杠命令可能变化：命令名、输出内容和支持范围都有可能随版本演进。
- `exec` 和 `review` 的参数可能增加或调整。
- 沙箱和审批策略的默认行为可能变化，尤其是新版本加入新策略时。
- 最可靠的检查方式，始终是你本地的 `codex --help` 和对应子命令帮助。

25.1 最推荐的检查顺序

1. 先看 `codex --help`；
2. 再看 `codex <subcommand> --help`；
3. 再回到官方文档页面查对应章节；
4. 最后再参考博客、截图、旧教程。

25.2 哪些地方最容易发生版本差异

- 首次登录流程；
- 斜杠命令列表；
- `codex exec` 的自动化参数；
- `review` 的输出样式；
- 审批与沙箱策略的默认组合。

小结

当版本变化发生时，不要先怀疑自己记错了，也不要先怀疑教程错了。先看本地 help，这是最快的判断方法。

26 FAQ

这一节学完你能做什么

学完这一节，你会对几个最常见的新手问题有直接答案，尤其是“上下文窗口超了怎么办”这种很容易卡住的问题。

26.1 Q1：如果上下文窗口超了，应该怎么办？

最实用的做法不是“硬塞更多内容”，而是**主动压缩上下文**。可以按这个顺序处理：

1. 先缩小任务范围，只围绕一个功能、一个报错或一组文件继续问。
2. 用 `/compact` 压缩长会话，把已经确认的背景收成短摘要。
3. 用 `/mention` 或直接点名文件路径，只把真正关键的文件拉进当前上下文。
4. 如果当前话题已经太散，直接开一轮新会话，并把上一轮结论用 3 到 5 句话重新总结后贴进去。

提示

判断标准很简单：如果你已经开始反复贴长日志、长代码、长历史对话，就说明该收缩上下文了。先总结，再继续，通常比硬接着聊更稳。

26.2 Q2：它回答得很空泛，是模型不行吗？

大多数时候不是。更常见的原因是任务太宽、目标不够具体，或者没有要求它按文件、按步骤、按输出格式回答。先把问题缩到“哪个功能、哪几个文件、最后要什么格式”，效果通常会马上变好。

26.3 Q3：什么时候该开新会话，而不是一直接着问？

当你发现当前会话已经同时混进了多个目标，比如“看项目结构、修 bug、改文案、做 CI”，这时就该拆。一个会话尽量只服务一个主目标，后续会更稳，也更容易复盘。

26.4 Q4：它改完代码后，我最少要做什么检查？

最少做三件事：

1. 看 `/diff`，确认改动范围对不对；
2. 跑最相关的测试、构建或 lint；
3. 自己走一遍最关键的用户路径。

小结

FAQ 里最值得先记的一条就是：上下文一旦开始发散，就收缩任务、压缩会话、必要时新开一轮。不要指望靠堆更多上下文来解决所有问题。

27 官方资源与版本说明

这一节学完你能做什么

学完这一节，你会知道这份教程背后的主要参考来源，也知道当本地行为和教程不一致时应该信谁。

这份教程的关键事实主要来自两类来源：

- 本地 CLI 帮助输出：`codex --help`、`codex exec --help`、`codex review --help`、`codex login --help` 等。
- OpenAI 官方 Codex 文档。

建议优先查看这些官方页面：

- CLI 总览：https://developers.openai.com/codex/cli
- 认证：https://developers.openai.com/codex/auth
- 审批与安全：https://developers.openai.com/codex/agent-approvals-security
- 命令行选项：https://developers.openai.com/codex/cli/reference
- 斜杠命令：https://developers.openai.com/codex/cli/slash-commands
- 非交互模式：https://developers.openai.com/codex/noninteractive
- Best practices：https://developers.openai.com/codex/learn/best-practices

27.1 当教程和你本地行为不一致时怎么办

优先级建议这样排：

1. 先看你本地的 `codex --help`；
2. 再看对应的官方文档页面；
3. 最后再看第三方博客、帖子或旧截图。

进阶阅读

原因很简单：Codex CLI 更新比较快，而第三方教程经常会落后于当前版本。

小结

真正可依赖的来源，始终是“你机器上的 help 输出 + OpenAI 官方 Codex 文档”。这两者要优先于任何二手资料。

28 一页式总结页

1. 先做什么
进入项目目录，确认位置没错。

2. 先跑什么
先看 `codex --help`，再登录，再做最小验证。

3. 日常怎么用
交互式用 `codex`，边问边收紧范围。

4. 改完怎么查
先 `/review`，再 `/diff`，再自己验证。

5. 权限怎么选
本地优先 `workspace-write`，别急着全放开。

6. 一句话记忆
说清楚任务，小步推进，看状态，审改动，做验证。